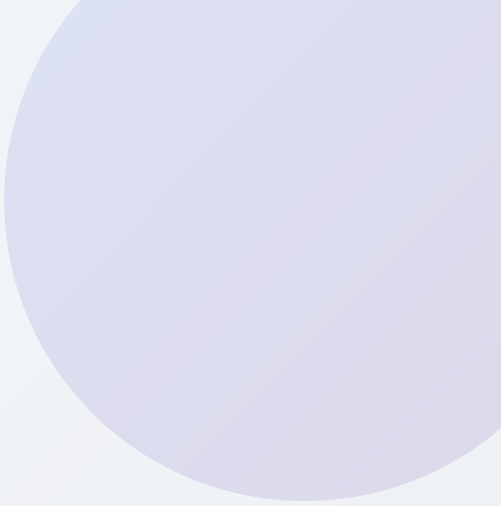


Greedy & Dynamic Programming Algorithms



Complete University Study Notes with Proofs, Complexity Analysis, and Visual Walkthroughs

Based on Kleinberg & Tardos, CLRS, and Standard Course Curriculum

Comprehensive Reference for University Examinations



Table of Contents

Part I: Greedy Algorithms

1. Interval Scheduling: The Greedy Algorithm Stays Ahead
 - 1.1 Algorithm and Working
 - 1.2 Time Complexity Analysis
 - 1.3 Visual Walkthrough
 - 1.4 Theorem 4.1: Compatible Set
 - 1.5 Theorem 4.2: Stays Ahead Property
 - 1.6 Theorem 4.3: Optimality Proof
2. Interval Partitioning (Depth/Classroom Scheduling)
3. Fractional Knapsack Problem
4. 0-1 Knapsack: Why Greedy Fails
5. Elements of the Greedy Strategy (CLRS)

Part II: Dynamic Programming

6. Weighted Interval Scheduling
 - 6.1 Problem Definition and $p(j)$
 - 6.2 The Recurrence Relation
 - 6.3 Naive Recursive Algorithm
 - 6.4 Memoization (M-Compute-Opt)
 - 6.5 Iterative Bottom-Up
 - 6.6 Finding the Solution Set
7. Subset Sum Problem
8. 0-1 Knapsack (Dynamic Programming)
9. Principles of Dynamic Programming

Summary: Algorithm Comparison

PART I

Greedy Algorithms

1. Interval Scheduling: The Greedy Algorithm Stays Ahead

1.1 Problem Definition

Given a set of n requests, where each request i corresponds to a time interval with start time $s(i)$ and finish time $f(i)$, the **Interval Scheduling Problem** asks us to select a **maximum-size compatible subset** of intervals. Two intervals are **compatible** if they do not overlap in time.

Key Difference from Weighted Version: In the standard interval scheduling problem, all intervals have equal value (implicitly 1), so we maximize the *count* of intervals. In the weighted version (covered in Part II), each interval has a value v_i and we maximize total value.

1.2 The Greedy Algorithm

Algorithm: Greedy Interval Scheduling

```
Sort intervals by finish time in nondecreasing order
Let  $I_1, I_2, \dots, I_n$  be the sorted intervals (so  $f(i) \leq f(j)$  when  $i < j$ )
 $A$  = empty set
For  $j = 1, 2, \dots, n$ 
    If  $I_j$  is compatible with all intervals in  $A$  then
        Add  $I_j$  to  $A$ 
    Endif
Endfor
Return  $A$ 
```

Compatibility Check: Since intervals in A are sorted by finish time and all mutually compatible, we only need to check if the start time of I_j is at least the finish time of the *last* interval added to A :

$$s(j) \geq f(\text{last}(A)) \quad (1.1)$$

1.3 Working of the Algorithm

The algorithm operates in two phases:

1. **Sorting Phase:** All n intervals are sorted by finish time in ascending order. This takes $O(n \log n)$ time.
2. **Selection Phase:** We scan through the sorted intervals exactly once. We always select the first interval (it has the earliest finish time). For each subsequent interval, we check if it starts at or after the finish time of the most recently selected interval. If yes, we select it; otherwise, we skip it.

Why this greedy choice works: By always selecting the interval with the *earliest finish time* among all compatible options, we leave the maximum possible remaining time for future intervals. This greedy choice property is what makes the algorithm optimal.

1.4 Time Complexity Analysis (Full Derivation)

Theorem: The greedy interval scheduling algorithm runs in $O(n \log n)$ time.

Proof of Time Complexity:

Phase 1 — Sorting: Using an efficient comparison-based sorting algorithm (Merge Sort or Heap Sort), sorting n intervals by finish time requires $O(n \log n)$ comparisons in the worst case. Each comparison of finish times takes $O(1)$ time. Therefore:

$$T_{\text{sort}}(n) = O(n \log n)$$

Phase 2 — Selection: After sorting, the selection phase makes a single pass through the n intervals. For each interval I_j , the compatibility check compares $s(j)$ with the finish time of the last selected interval:

$$T_{\text{check}}(j) = O(1)$$

Since this is done for all n intervals:

$$T_{\text{selection}}(n) = \sum_{j=1}^n O(1) = O(n)$$

Total Time Complexity:

$$T(n) = T_{\text{sort}}(n) + T_{\text{selection}}(n) = O(n \log n) + O(n) = O(n \log n)$$

The $O(n \log n)$ term dominates, giving the overall complexity.

Space Complexity: $O(n)$ to store the sorted intervals and the selected set A.

1.5 Visual Walkthrough with Example

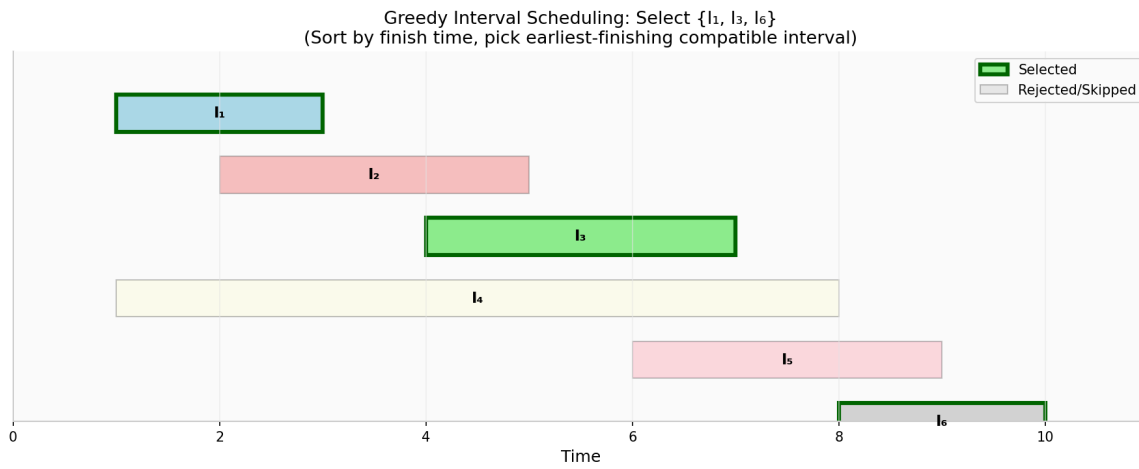


Figure 1.1 Greedy Interval Scheduling execution. Top row shows all intervals; green-bordered intervals are selected by the greedy algorithm (earliest finish time first).

Detailed Step-by-Step Trace:

Table 1.1 Greedy Selection Trace for Example Instance

Step	Consider	Start	Finish	Last in A	Compatible?	Action	A after
1	I_1	1	3	—	Yes (A empty)	Add I_1	$\{I_1\}$
2	I_2	2	5	I_1 (finishes 3)	No ($2 < 3$)	Skip	$\{I_1\}$
3	I_3	4	7	I_1 (finishes 3)	Yes ($4 \geq 3$)	Add I_3	$\{I_1, I_3\}$
4	I_5	6	9	I_3 (finishes 7)	No ($6 < 7$)	Skip	$\{I_1, I_3\}$
5	I_6	8	10	I_3 (finishes 7)	Yes ($8 \geq 7$)	Add I_6	$\{I_1, I_3, I_6\}$
6	I_4	1	8	I_6 (finishes 10)	No ($1 < 10$)	Skip	$\{I_1, I_3, I_6\}$

Result: $A = \{I_1, I_3, I_6\}$ with $|A| = 3$ intervals. **Verification of optimality:** Intervals I_1, I_2, I_4 all overlap at time 2.5-3, so at most one can be chosen. Similarly I_3 and I_5 overlap. Thus no solution can contain more than 3 intervals.

1.6 Theorem 4.1: The Greedy Set is Compatible

Theorem 4.1

The set A produced by the greedy algorithm is a compatible set of requests (no two intervals in A overlap).

Proof:

By construction, when the algorithm considers interval I_j , it adds I_j to A only if $s(j) \geq f(\text{last}(A))$, where $\text{last}(A)$ is the interval most recently added to A . Since intervals are processed in order of nondecreasing finish time, all intervals already in A finish no later than $\text{last}(A)$. Therefore, I_j starts at or after all intervals in A finish, meaning I_j is compatible with *every* interval in A . By induction on the number of intervals added, all pairs in A are compatible. ■

1.7 Theorem 4.2: The "Stays Ahead" Property

Theorem 4.2

Let $A = \{i_1, i_2, \dots, i_k\}$ be the intervals selected by the greedy algorithm in order. Let $O = \{j_1, j_2, \dots, j_m\}$ be any optimal solution, also ordered by finish time. Then for all indices $r \leq k$:

$$f(i_r) \leq f(j_r) \quad (1.2)$$

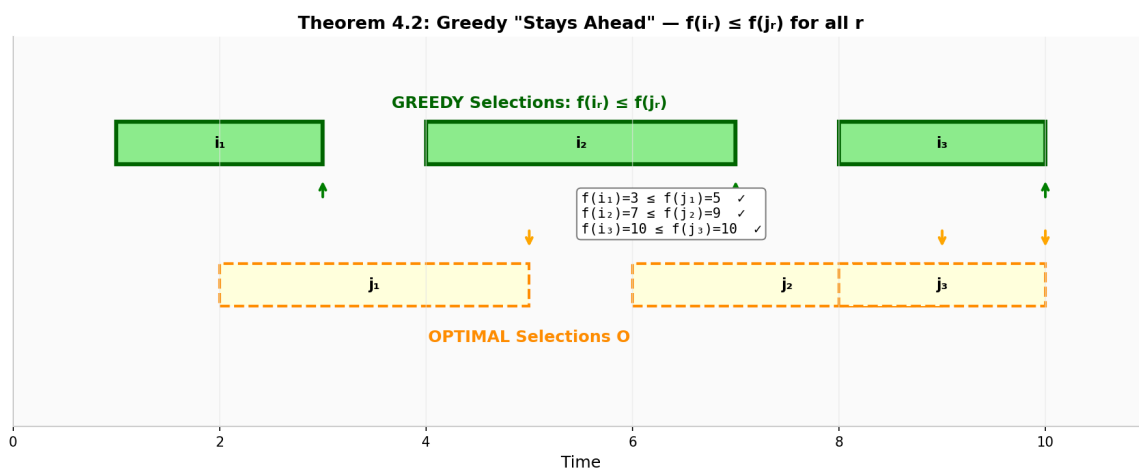


Figure 1.2 The "Stays Ahead" property: each greedy interval finishes no later than the corresponding optimal interval.

Proof (by induction on r):

Base Case ($r = 1$): The greedy algorithm starts by selecting the interval with the *minimum finish time* among all intervals. Therefore, $f(i_1) \leq f(j_1)$ for any interval j_1 in any optimal solution. The base case holds. ✓

Inductive Hypothesis: Assume the statement is true for $r - 1$, i.e., $f(i_{r-1}) \leq f(j_{r-1})$.

Inductive Step (prove for r):

Since O consists of compatible intervals, we know that:

$$f(j_{r-1}) \leq s(j_r)$$

This is because j_r must start after j_{r-1} finishes (they are compatible).

From the inductive hypothesis:

$$f(i_{r-1}) \leq f(j_{r-1})$$

Combining these two inequalities by transitivity:

$$f(i_{r-1}) \leq f(j_{r-1}) \leq s(j_r)$$

Therefore:

$$f(i_{r-1}) \leq s(j_r)$$

This means that interval j_r starts after interval i_{r-1} finishes. So when the greedy algorithm was selecting its r -th interval, j_r was still available in the set R of intervals that don't overlap with any previously selected interval.

The greedy algorithm always selects the available interval with the **smallest finish time**. Since j_r was available at that time, the greedy algorithm's choice i_r must have finish time no later than j_r :

$$f(i_r) \leq f(j_r)$$

This completes the induction step. ■

1.8 Theorem 4.3: The Greedy Algorithm is Optimal

Theorem 4.3

The greedy algorithm returns an optimal set A .

Proof (by contradiction):

Assume for contradiction that A is not optimal. Then there exists an optimal set O with strictly more intervals than A , i.e., $m > k$ where $|A| = k$ and $|O| = m$.

Apply Theorem 4.2 with $r = k$:

$$f(i_k) \leq f(j_k)$$

Since $m > k$, there exists an interval j_{k+1} in O . Because O is a set of compatible intervals:

$$f(j_k) \leq s(j_{k+1})$$

Combining with the above by transitivity:

$$f(i_k) \leq f(j_k) \leq s(j_{k+1})$$

This means j_{k+1} starts after i_k finishes, so j_{k+1} is compatible with all intervals in $A = \{i_1, i_2, \dots, i_k\}$. Therefore, j_{k+1} should still be in the set R of available intervals when the greedy algorithm finished selecting i_k .

But the greedy algorithm stopped after selecting k intervals, which only happens when R is empty (no more compatible intervals available). This is a **contradiction**.

Therefore, our assumption that A is not optimal must be false. The greedy algorithm returns an optimal set. ■

Summary: Interval Scheduling Greedy Algorithm

- **Algorithm:** Sort by finish time, then greedily pick earliest-finishing compatible interval.
- **Time Complexity:** $O(n \log n)$ — dominated by sorting.
- **Space Complexity:** $O(n)$.
- **Proof Technique:** "Stays Ahead" — greedy always finishes no later than optimal.
- **Key Theorems:** Compatible set (4.1), Stays Ahead (4.2), Optimality (4.3).

2. Interval Partitioning (Depth / Classroom Scheduling)

2.1 Problem Definition

The **Interval Partitioning Problem** (also called Interval Coloring or Classroom Scheduling) asks: given a set of intervals, assign a label (color/resource) to each interval such that no two overlapping intervals share the same label. The goal is to minimize the number of labels used.

Definition: Depth of an Interval Set

The **depth** of a set of intervals is the maximum number of intervals that pass over any single point on the time-line. This is a lower bound on the number of labels needed.

2.2 The Greedy Algorithm

Algorithm: Interval Partitioning (Greedy by Start Time)

```
Sort the intervals by their start times, breaking ties arbitrarily
Let  $I_1, I_2, \dots, I_n$  denote the intervals in this order
For  $j = 1, 2, \dots, n$ 
    For each interval  $I_i$  that precedes  $I_j$  in sorted order and overlaps
    it
        Exclude the label of  $I_i$  from consideration for  $I_j$ 
    Endfor
    If there is any label from  $\{1, 2, \dots, d\}$  that has not been excluded
    then
        Assign a non-excluded label to  $I_j$  (smallest available)
    Else
        Leave  $I_j$  unlabeled (need a new label)
    Endif
Endfor
```

2.3 Working of the Algorithm

By processing intervals in order of increasing start time, when we reach interval I_j , we have complete information about all intervals that could possibly conflict with it (any interval that starts later cannot overlap with I_j 's start). The algorithm always tries to reuse an existing label before introducing a new one.

2.4 Time Complexity Analysis (Full Derivation)

Theorem: The interval partitioning algorithm runs in $O(n^2)$ time (naive) or $O(n \log n)$ with a priority queue.

Naive Analysis:

Phase 1 — Sorting: $O(n \log n)$ by start time.

Phase 2 — Label Assignment: For each of the n intervals, in the worst case we may need to check all $O(n)$ preceding intervals to find overlaps and excluded labels. Checking compatibility of two intervals takes $O(1)$ time. Therefore:

$$T_{\text{assign}}(n) = \sum_{j=1}^n O(j) = O(n^2)$$

Total (naive):

$$T(n) = O(n \log n) + O(n^2) = O(n^2)$$

Optimized with Priority Queue: Maintain a min-heap of labels with their "release times" (when the interval using that label finishes). For each new interval, extract all labels whose release time $\leq$ current start time (they're now free). Assign the smallest available label. Heap operations take $O(\log d)$ where d is the number of labels. Since $d \leq n$:

$$T_{\text{optimized}}(n) = O(n \log n) + O(n \log n) = O(n \log n)$$

2.5 Visual Walkthrough

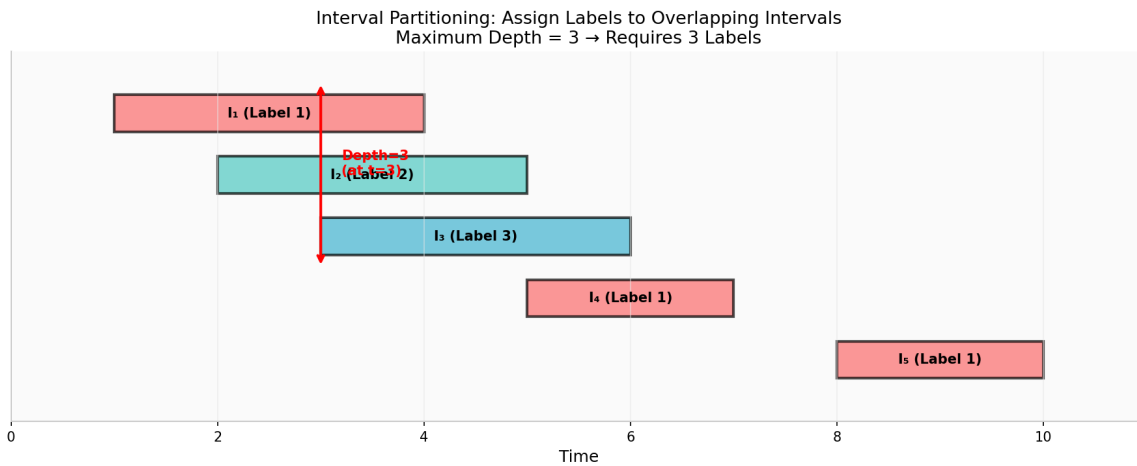


Figure 2.1 Interval partitioning: maximum depth = 3 requires 3 labels. The greedy algorithm processes by start time and assigns the smallest available label.

2.6 Proof of Correctness

Theorem

The greedy interval partitioning algorithm uses exactly d labels, where d is the depth of the interval set. This is optimal.

Proof:

Upper bound: We show the algorithm never needs more than d labels. When processing interval I_j , at most $d - 1$ intervals can overlap with it at any point (by definition of depth). These are the only intervals that could exclude labels. Since there are d labels total, at least one label is always available.

Lower bound: At the point where depth d is achieved, d intervals simultaneously overlap. Each needs a distinct label. Thus any valid labeling needs at least d labels.

Since the algorithm uses at most d labels and at least d labels are necessary, the algorithm is optimal. ■

3. Fractional Knapsack Problem

3.1 Problem Definition

Given n items, where item i has weight w_i and value v_i , and a knapsack with capacity W , the **Fractional Knapsack Problem** asks us to select fractions of items to maximize total value without exceeding weight capacity. Unlike 0-1 knapsack, we can take any fraction $x_i \in [0, 1]$ of each item.

3.2 The Greedy Algorithm

Algorithm: Fractional Knapsack (Greedy by Density)

```

Sort the items by value per pound ( $v_i / w_i$ ) in decreasing order
Let  $a_1, a_2, \dots, a_n$  denote the items in this order
Let  $W$  be the knapsack capacity
Set  $total\_value = 0$ 
Set  $remaining\_capacity = W$ 
For  $j = 1, 2, \dots, n$ 
    If  $remaining\_capacity = 0$  then Break
    If  $w_j \leq remaining\_capacity$  then
        Take all of item  $a_j$ 
         $total\_value = total\_value + v_j$ 
         $remaining\_capacity = remaining\_capacity - w_j$ 
    Else
        Take fraction =  $remaining\_capacity / w_j$  of item  $a_j$ 
         $total\_value = total\_value + fraction * v_j$ 
         $remaining\_capacity = 0$ 
    Endif
Endfor
Return  $total\_value$ 

```

3.3 Working of the Algorithm

1. **Sorting Phase:** Compute the value-to-weight ratio (density) $r_i = v_i / w_i$ for each item. Sort items in decreasing order of this ratio. This takes $O(n \log n)$ time.
2. **Greedy Selection:** Process items in sorted order. For each item, if it fits entirely, take all of it. If it doesn't fit, take exactly enough to fill the remaining capacity.

Why this works: The greedy strategy ensures every unit of capacity is filled with the maximum possible value density. Since we can take fractions, we never "waste" capacity.

3.4 Time Complexity Analysis (Full Derivation)

Theorem: The fractional knapsack algorithm runs in $O(n \log n)$ time.

Phase 1 — Computing Ratios: For each of n items, compute v_i/w_i . Division of two numbers takes $O(1)$ time.

$$T_{\text{ratios}}(n) = \sum_{i=1}^n O(1) = O(n)$$

Phase 2 — Sorting by Ratio: Sort n items by their density values. Using Merge Sort or Heap Sort:

$$T_{\text{sort}}(n) = O(n \log n)$$

Phase 3 — Selection: Single pass through n items. Each item requires $O(1)$ work (comparison, arithmetic, update):

$$T_{\text{selection}}(n) = \sum_{j=1}^n O(1) = O(n)$$

Total Time Complexity:

$$T(n) = O(n) + O(n \log n) + O(n) = O(n \log n)$$

Space Complexity: $O(n)$ to store items, their ratios, and the sorted order.

3.5 Visual Walkthrough

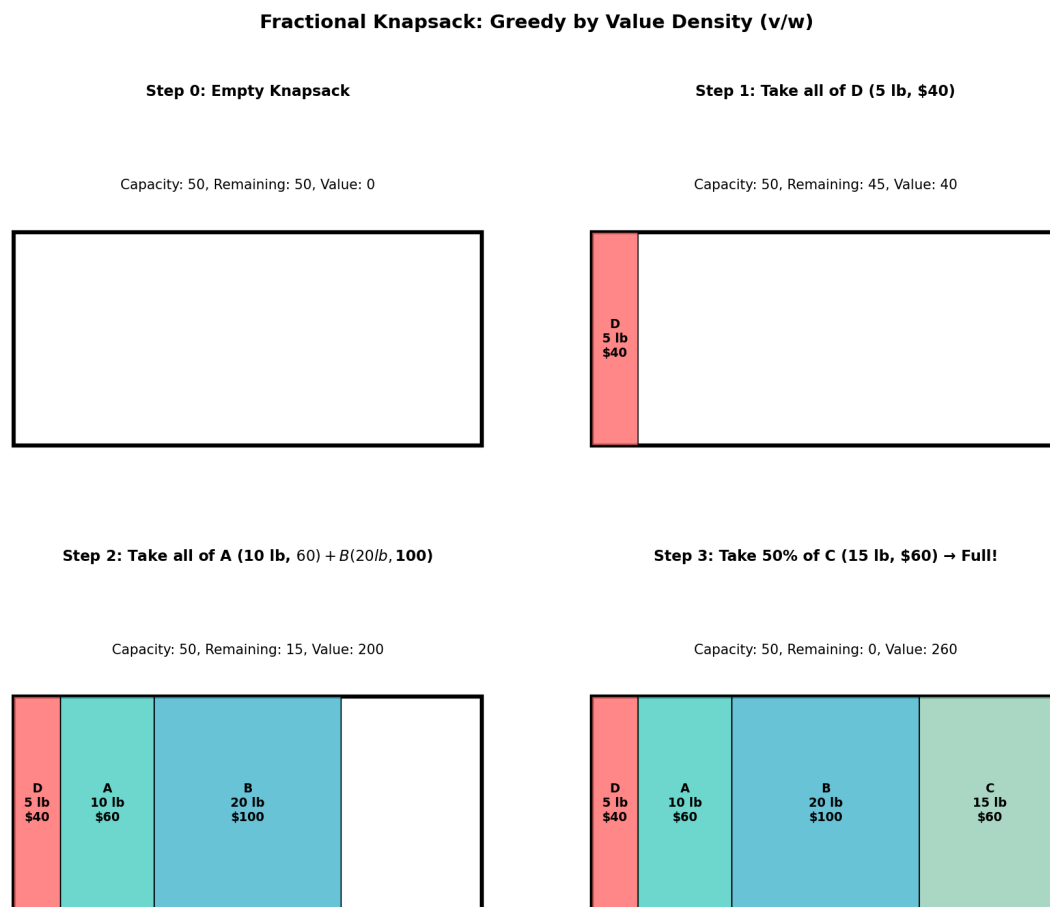


Figure 3.1 Fractional knapsack execution: items sorted by value density (D: $8/lb$, A: $6/lb$, B: $5/lb$, C: $4/lb$). The knapsack is filled greedily, taking fractions of the last item if needed.

Detailed Trace:

Table 3.1 Fractional Knapsack Execution Trace (W = 50)

Step	Item	Ratio	Action	Weight Taken	Value Added	Remaining	Total Value
1	D	8.0	Take all	5	\$40	45	\$40
2	A	6.0	Take all	10	\$60	35	\$100
3	B	5.0	Take all	20	\$100	15	\$200
4	C	4.0	Take $15/30 = 0.5$	15	\$60	0	\$260

Proof (Exchange Argument):

Let the items be sorted by value density: $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$.

Let $X = (x_1, x_2, \dots, x_n)$ be the solution produced by the greedy algorithm, where $x_i \in [0, 1]$ is the fraction of item i taken.

Let $Y = (y_1, y_2, \dots, y_n)$ be any optimal solution. Suppose for contradiction that Y is strictly better than X .

Then there must exist some item i where $y_i > x_i$ and some item j where $y_j < x_j$, with the density of item i less than the density of item j (i.e., $i > j$ in sorted order, meaning $v_i/w_i < v_j/w_j$).

Consider transferring a small amount δ of weight from item i to item j in solution Y :

- Remove δ weight of item i : lose value $\delta \cdot (v_i/w_i)$
- Add δ weight of item j : gain value $\delta \cdot (v_j/w_j)$

Since $v_j/w_j > v_i/w_i$, the net change is:

$$\delta \cdot \left(\frac{v_j}{w_j} - \frac{v_i}{w_i} \right) > 0$$

This means Y was not optimal — we can improve it by shifting weight from lower-density to higher-density items. Repeating this argument, any optimal solution must allocate weight in order of decreasing density, which is exactly what the greedy algorithm does. ■

4. 0-1 Knapsack: Why Greedy Fails

4.1 Problem Definition

In the **0-1 Knapsack Problem**, each item must be taken whole or not at all. The thief cannot take fractional amounts. This seemingly small change makes the greedy strategy fail.

4.2 The Counterexample

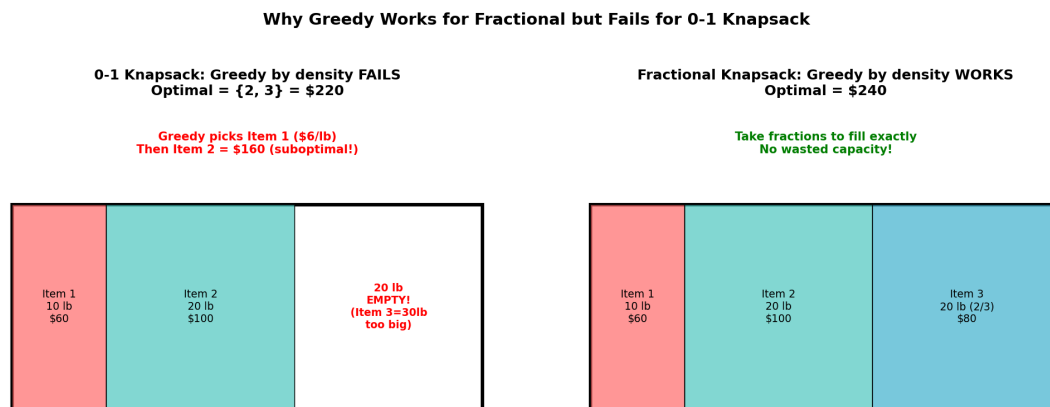


Figure 4.1 0-1 knapsack (left) vs fractional knapsack (right) with the same instance. Greedy by density fails for 0-1 because taking Item 1 first leaves empty space that cannot be efficiently filled.

Table 4.1 0-1 Knapsack Counterexample: Capacity $W = 50$

Item	Weight	Value	Density (\$/lb)
1	10	\$60	\$6.00
2	20	\$100	\$5.00
3	30	\$120	\$4.00

Greedy by density: Take Item 1 (6/lb), then Item 2 (5/lb), then cannot fit Item 3 (30 lb > 20 lb remaining). Total value = **\$160**.

Optimal 0-1 solution: Take Items 2 and 3 (20 + 30 = 50 lb). Total value = **\$220**.

Key Insight: Taking the highest-density item first may leave the knapsack with unused capacity that cannot be efficiently filled by remaining items. The "empty space" lowers the effective value per pound of the load.

4.3 Why Greedy Works for Fractional but Not 0-1

Table 4.2 Fractional vs 0-1 Knapsack

Aspect	Fractional Knapsack	0-1 Knapsack
Decision	Take any fraction $x_i \in [0, 1]$	Binary: take or leave
Greedy works?	Yes — always fill with highest density	No — may leave gaps
Empty space	Never — fill exactly to W with fractions	Can occur — items don't fit perfectly
Solution approach	Greedy $O(n \log n)$	Dynamic Programming $O(nW)$
Why empty space matters	Not applicable	Taking Item 1 (10 lb) leaves 20 lb gap. Item 3 (30 lb) doesn't fit, so space is wasted.

Mathematical Explanation: In 0-1 knapsack, when deciding to include item i , we must *compare* the solution to the subproblem *including* the item with the solution to the subproblem *excluding* the item *before* making the choice. This gives rise to **many overlapping subproblems** — a hallmark of dynamic programming. The greedy algorithm makes a local choice without this global comparison, which can be suboptimal.

5. Elements of the Greedy Strategy (CLRS Framework)

5.1 Core Definition

A **greedy algorithm** obtains an optimal solution to a problem by making a **sequence of choices**. At each decision point, the algorithm makes the choice that **seems best at the moment** (locally optimal). This heuristic does not always produce an optimal solution, but sometimes it does.

5.2 Two Methodologies for Developing Greedy Algorithms

Method 1: The Dynamic-Programming-First Approach

Six-step process:

1. Determine the **optimal substructure** of the problem.
2. Develop a **recursive solution**.
3. Show that if you make the **greedy choice**, then only **one subproblem** remains.
4. **Prove** that it is always safe to make the greedy choice.
5. Develop a **recursive algorithm** that implements the greedy strategy.
6. Convert the recursive algorithm to an **iterative algorithm**.

Used in Section 15.1 for the activity-selection problem.

Method 2: The Direct/Greedy-First Approach

Three-step process:

1. **Cast the optimization problem** as one in which you make a choice and are left with **one subproblem** to solve.
2. **Prove** that there is always an **optimal solution** to the original problem that makes the greedy choice, so that the greedy choice is **always safe**.
3. **Demonstrate optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that if you combine an **optimal solution to the subproblem** with the **greedy choice**, you arrive at an **optimal solution to the original problem**.

This is the more typical approach.

Important Caveat

Beneath every greedy algorithm, there is **almost always** a more cumbersome dynamic-programming solution.

5.3 The Two Key Ingredients

No universal method works all the time. The two key ingredients for greedy algorithms are:

1. **Greedy-choice property:** You can assemble a globally optimal solution by making locally optimal (greedy) choices. The choice depends on choices so far, but **cannot depend on future choices or solutions to subproblems**.
2. **Optimal substructure:** An optimal solution to the problem contains within it optimal solutions to subproblems.

5.4 Greedy vs Dynamic Programming

Table 5.1 Greedy Algorithms vs Dynamic Programming

Aspect	Dynamic Programming	Greedy Algorithm
Choice dependency	Choice usually depends on solutions to subproblems	Choice depends on choices so far , not future subproblems
Order of solving	Typically bottom-up	Typically top-down
Memoization	Can solve top-down with memoization	Makes choice before solving any subproblems
Number of subproblems	May need to solve many subproblems	Only one subproblem remains after greedy choice
Proof requirement	Optimal substructure + overlapping subproblems	Greedy-choice property + optimal substructure
Efficiency	Generally slower (polynomial but higher degree)	Often faster (can be linear or linearithmic)

PART II

Dynamic Programming

6. Weighted Interval Scheduling

6.1 Problem Definition

The **Weighted Interval Scheduling Problem** is a strictly more general version of interval scheduling where each interval i has a **value/weight** v_i , and we want to select a compatible subset with **maximum total value** (not just maximum count). The standard problem is the special case where all $v_i = 1$.

Why Greedy Fails for Weighted Version

Most greedy algorithms that work for standard interval scheduling **do not** solve the weighted version optimally. Even the previously optimal greedy approach (repeatedly choosing the interval that ends earliest) **fails** in the weighted setting. Consider: three intervals with values 1, 3, and 1. Greedy by earliest finish picks intervals 1 and 3 (total value 2), but the optimal is interval 2 alone (value 3).

6.2 Notation and Setup

- n requests labeled $1, 2, \dots, n$
- Each request i : start time s_i , finish time f_i , value v_i
- Requests sorted by **nondecreasing finish time**: $f_1 \leq f_2 \leq \dots \leq f_n$
- **Compatibility**: Two intervals are compatible if they do not overlap
- **Goal**: Select subset $S \subseteq \{1, \dots, n\}$ of mutually compatible intervals to maximize $\sum_{i \in S} v_i$

6.3 The $p(j)$ Function

Definition: $p(j)$

For interval j , $p(j)$ is the **largest index** $i < j$ such that intervals i and j are **disjoint** (non-overlapping). If no such interval exists, $p(j) = 0$.

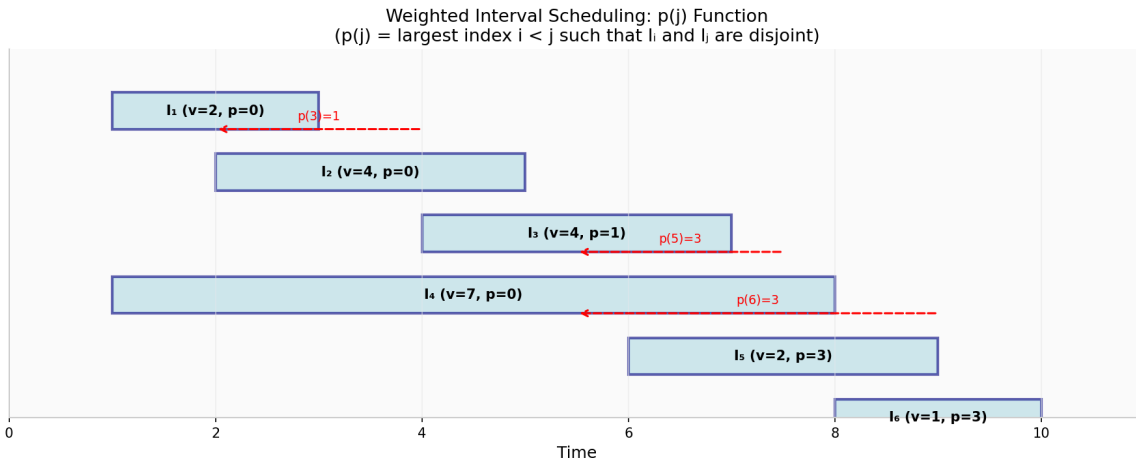


Figure 6.1 The $p(j)$ function: dashed red arrows show the last compatible interval before each j . For example, $p(5) = 3$ because I_3 is the rightmost interval that finishes before I_5 starts.

Computing $p(j)$ efficiently: Since intervals are sorted by finish time, we can find $p(j)$ for all j in $O(n \log n)$ using binary search, or $O(n^2)$ naively by checking all previous intervals.

6.4 The Recurrence Relation

Theorem 6.1 (Recurrence)

For each $j = 1, 2, \dots, n$:

$$OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1)) \quad (6.1)$$

Proof (by case analysis):

Consider an optimal solution O_j for intervals $\{1, \dots, j\}$. There are exactly two cases:

Case 1: $j \in O_j$ (interval j is selected).

By definition of $p(j)$, intervals $p(j) + 1, p(j) + 2, \dots, j - 1$ all overlap interval j . Therefore, **none** of these can be in O_j . The remaining selected intervals must come from $\{1, \dots, p(j)\}$. Moreover, O_j must include an **optimal solution** to $\{1, \dots, p(j)\}$ — otherwise we could replace it with a better one without conflicting with j .

$$\text{Value} = v_j + \text{OPT}(p(j))$$

Case 2: $j \notin O_j$ (interval j is not selected).

Then O_j is simply the optimal solution to $\{1, \dots, j - 1\}$.

$$\text{Value} = \text{OPT}(j - 1)$$

Since these are the only two possibilities, the optimal value is the maximum of these two cases. ■

Theorem 6.2 (Decision Rule)

Request j belongs to an optimal solution on $\{1, 2, \dots, j\}$ if and only if:

$$v_j + \text{OPT}(p(j)) \geq \text{OPT}(j - 1) \quad (6.2)$$

6.5 Naive Recursive Algorithm: Compute-Opt

Algorithm: Compute-Opt (Naive Recursive)

```

Compute-Opt(j)
  If j = 0 then
    Return 0
  Else
    Return max(v_j + Compute-Opt(p(j)), Compute-Opt(j - 1))
  Endif
End

Invoke Compute-Opt(n) to obtain OPT(n)

```

Theorem 6.3 (Correctness of Compute-Opt)

Compute-Opt(j) correctly computes $\text{OPT}(j)$ for each $j = 1, 2, \dots, n$.

Proof (by strong induction on j):

Base Case ($j = 0$): Compute-Opt(0) returns 0, and by definition $OPT(0) = 0$. Correct. ✓

Inductive Hypothesis: Assume Compute-Opt(i) correctly computes $OPT(i)$ for all $i < j$.

Inductive Step (prove for j):

Compute-Opt(j) returns:

$$\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j - 1))$$

By the inductive hypothesis:

- $\text{Compute-Opt}(p(j)) = OPT(p(j))$ since $p(j) < j$
- $\text{Compute-Opt}(j - 1) = OPT(j - 1)$ since $j - 1 < j$

Therefore:

$$\text{Compute-Opt}(j) = \max(v_j + OPT(p(j)), OPT(j - 1)) = OPT(j)$$

The last equality follows from Theorem 6.1. Thus Compute-Opt(j) correctly computes $OPT(j)$. ■

6.6 Why the Naive Algorithm is Exponential

Theorem: The running time of naive Compute-Opt is $O(2^n)$.

Full Derivation:

Let $T(j)$ be the number of recursive calls made by Compute-Opt(j). From the recurrence:

$$T(j) = T(p(j)) + T(j - 1) + 1$$

Worst-case instance: Consider the case where $p(j) = j - 2$ for all $j = 2, 3, \dots, n$. This creates the recurrence:

$$T(j) = T(j - 2) + T(j - 1) + 1$$

This is **identical to the Fibonacci recurrence**. The Fibonacci numbers grow as $F_j \approx \phi^j / \sqrt{5}$ where $\phi = (1 + \sqrt{5})/2 \approx 1.618$.

Therefore:

$$T(n) = \Theta(\phi^n) = \Theta(1.618^n) \subseteq O(2^n)$$

Key insight: The exponential runtime is purely due to **redundant calls** to the same subproblems. Compute-Opt only solves $n + 1$ **distinct** subproblems: Compute-Opt(0), Compute-Opt(1), ..., Compute-Opt(n). But many subproblems are solved repeatedly.

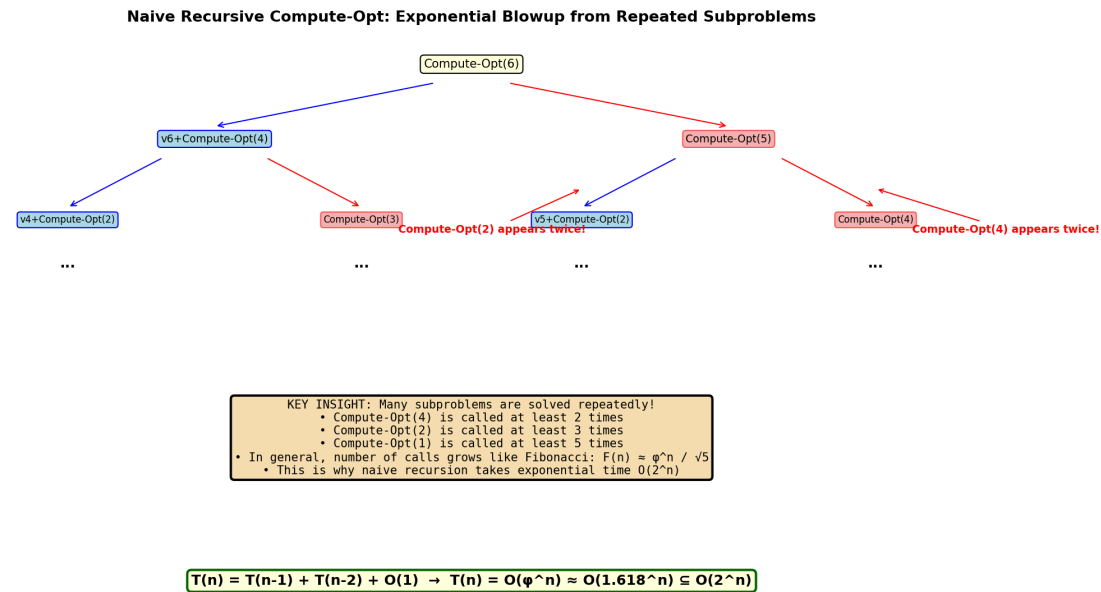


Figure 6.2 Recursion tree for Compute-Opt showing exponential blowup. Compute-Opt(4) and Compute-Opt(2) appear multiple times — this redundancy is what makes the algorithm exponential.

6.7 Memoization: M-Compute-Opt

The key insight is to **store computed values** in a globally accessible array $M[0 \dots n]$.

Algorithm: M-Compute-Opt (Memoized)

```

Initialize array  $M[0 \dots n]$  with all entries set to "empty"
Set  $M[0] = 0$ 

M-Compute-Opt( $j$ )
  If  $j = 0$  then
    Return 0
  Else if  $M[j]$  is not empty then
    Return  $M[j]$ 
  Else
    Define  $M[j] = \max(v_j + \text{M-Compute-Opt}(p(j)), \text{M-Compute-Opt}(j - 1))$ 
    Return  $M[j]$ 
  Endif
End

Invoke M-Compute-Opt( $n$ ) to obtain  $\text{OPT}(n)$ 

```

Theorem 6.4 (Time Complexity of Memoized Version)

The running time of $M\text{-Compute-Opt}(n)$ is $O(n)$ (assuming intervals are sorted by finish times and $p(j)$ is precomputed).

Proof (Progress Measure Argument):

Time per call: Each call to $M\text{-Compute-Opt}(j)$ (excluding recursive calls it generates) performs $O(1)$ work:

- Check if $j = 0$: $O(1)$
- Check if $M[j]$ is empty: $O(1)$
- Compute max and store in $M[j]$: $O(1)$

Number of calls: We use a **progress measure** — the number of entries in M that are not "empty."

- Initially: 0 filled entries (only $M[0]$ is set, which is given).
- Each time $M\text{-Compute-Opt}$ is called with $M[j]$ empty, it fills $M[j]$ with a new value.
- The key insight: $M[j]$ **can transition from "empty" to "filled" at most once**.
- Since M has only $n + 1$ entries, there are at most $n + 1$ "productive" calls that actually compute a new value.
- Calls on already-filled entries return immediately without generating new recursive calls.

Therefore, the total number of calls to $M\text{-Compute-Opt}$ is bounded by $O(n)$, and since each call does $O(1)$ work:

$$T(n) = O(n) \cdot O(1) = O(n)$$

■

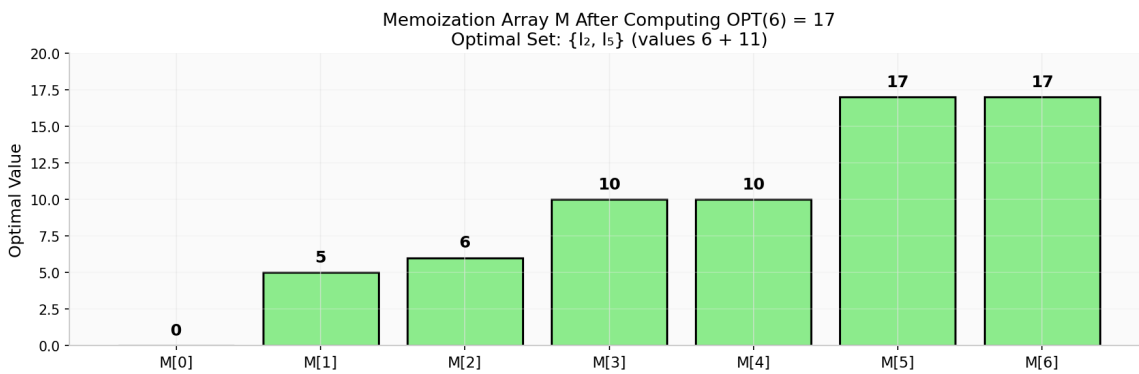


Figure 6.3 Memoization array M after computing $\text{OPT}(6)$. Each entry is filled exactly once, with $M[j]$ storing the optimal value for the first j intervals.

6.8 Iterative Bottom-Up Algorithm

Instead of memoized recursion, we can fill the array M **iteratively** from smallest to largest.

Algorithm: Iterative-Compute-Opt

```

Iterative-Compute-Opt
  M[0] = 0
  For j = 1, 2, ..., n
    M[j] = max(v_j + M[p(j)], M[j - 1])
  Endfor
  Return M[n]

```

Theorem (Correctness of Iterative Version)

Iterative-Compute-Opt correctly computes $OPT(n)$ in $O(n)$ time.

Proof (by induction on j):

Base Case: $M[0] = 0 = OPT(0)$. ✓

Inductive Step: Assume $M[i] = OPT(i)$ for all $i < j$. When computing $M[j]$, both $M[p(j)]$ and $M[j - 1]$ have already been computed (since $p(j) < j$ and $j - 1 < j$). By the inductive hypothesis, these equal $OPT(p(j))$ and $OPT(j - 1)$. Therefore:

$$M[j] = \max(v_j + OPT(p(j)), OPT(j - 1)) = OPT(j)$$

by Theorem 6.1. ✓

Running Time: The loop runs for n iterations, each doing $O(1)$ work:

$$T(n) = \sum_{j=1}^n O(1) = O(n)$$

■

Comparison: Memoization vs Iteration

Table 6.1 Memoized Recursion vs Bottom-Up Iteration

Aspect	Memoized (M-Compute-Opt)	Iterative (Iterative-Compute-Opt)
Direction	Top-down (starts from n)	Bottom-up (starts from 0)
Array filling	On-demand, only when needed	Systematic, all entries in order
Recursion depth	$O(n)$	None — purely iterative
Time complexity	$O(n)$	$O(n)$
Space complexity	$O(n)$ for M + recursion stack	$O(n)$ for M only
Code complexity	Slightly more complex	Simpler, more explicit

6.9 Finding the Actual Solution (Find-Solution)

So far we've only computed $OPT(n)$ — the **value** of the optimal solution. We want the **actual set of intervals**.

Algorithm: Find-Solution (Backtracking)

```

Find-Solution(j)
  If j = 0 then
    Output nothing
  Else
    If  $v_j + M[p(j)] \geq M[j - 1]$  then
      Output j together with the result of Find-Solution(p(j))
    Else
      Output the result of Find-Solution(j - 1)
    Endif
  Endif
End

Invoke Find-Solution(n) to obtain the optimal set

```

Theorem 6.5 (Correctness and Complexity of Find-Solution)

Given the array M of optimal values, Find-Solution returns an optimal solution in $O(n)$ time.

Proof:

Correctness (by induction on j):

Base case $j = 0$: Returns empty set, optimal with value 0. ✓

At step j , there are two cases:

Case 1: $v_j + M[p(j)] \geq M[j - 1]$

By Theorem 6.2, I_j is in some optimal solution. The optimal value $M[j] = v_j + M[p(j)]$. Find-Solution outputs j and recursively calls Find-Solution($p(j)$). By inductive hypothesis, this returns an optimal subset for $\{I_1, \dots, I_{p(j)}\}$ with value $M[p(j)]$. Adding I_j gives value $v_j + M[p(j)] = M[j]$. ✓

Case 2: $v_j + M[p(j)] < M[j - 1]$

Then $M[j] = M[j - 1]$, so the optimal solution doesn't include I_j . Find-Solution recursively calls Find-Solution($j - 1$). By inductive hypothesis, this returns an optimal subset for $\{I_1, \dots, I_{j-1}\}$ with value $M[j - 1] = M[j]$. ✓

Time Complexity:

Each call to Find-Solution(j) does $O(1)$ work and makes exactly one recursive call to either Find-Solution($p(j)$) or Find-Solution($j - 1$). Since $p(j) < j$ and $j - 1 < j$, the argument strictly decreases. Starting from n , we can make at most n recursive calls before reaching 0. Therefore:

$$T(n) = O(n)$$

■

6.10 Complete Example Trace

Table 6.2 Full Trace of Iterative-Compute-Opt and Find-Solution

j	I_j	$[s_j, f_j]$	v_j	$p(j)$	$v_j + M[p(j)]$	$M[j - 1]$	$M[j]$	Include?
1	I_1	[1, 3]	5	0	$5 + 0 = 5$	0	5	Yes
2	I_2	[2, 5]	6	0	$6 + 0 = 6$	5	6	Yes
3	I_3	[4, 7]	5	1	$5 + 5 = 10$	6	10	Yes
4	I_4	[6, 8]	4	2	$4 + 6 = 10$	10	10	Either
5	I_5	[5, 9]	11	2	$11 + 6 = 17$	10	17	Yes
6	I_6	[8, 10]	2	4	$2 + 10 = 12$	17	17	No

Find-Solution(6):

- At $j = 6$: $v_6 + M[p(6)] = 2 + 10 = 12 < M[5] = 17$. Skip I_6 . $\rightarrow$ Find-Solution(5)
- At $j = 5$: $v_5 + M[p(5)] = 11 + 6 = 17 \geq M[4] = 10$. Include I_5 . $\rightarrow$ Find-Solution(2)
- At $j = 2$: $v_2 + M[p(2)] = 6 + 0 = 6 \geq M[1] = 5$. Include I_2 . $\rightarrow$ Find-Solution(0)
- At $j = 0$: Done.

Optimal Set: $\{I_2, I_5\}$ with total value $6 + 11 = 17$. ✓

Summary: Weighted Interval Scheduling

Algorithm	Time	Space	Type
Naive Compute-Opt	$O(2^n)$	$O(n)$	Recursive
M-Compute-Opt	$O(n)$	$O(n)$	DP (Memoization)
Iterative-Compute-Opt	$O(n)$	$O(n)$	DP (Bottom-up)
Find-Solution	$O(n)$	$O(n)$	Backtracking

7. Subset Sum Problem

7.1 Problem Definition

Given n items with weights $w_1, w_2, \dots, w_n$ and a target weight W , find a subset of items whose total weight is **as large as possible without exceeding W** .

Formal Definition

Find subset $S \subseteq \{1, \dots, n\}$ maximizing $\sum_{i \in S} w_i$ subject to $\sum_{i \in S} w_i \leq W$.

7.2 Why the "Obvious" Subproblems Fail

The natural first attempt is to define $OPT(i)$ as the best solution using only the first i items. But this fails:

- **Case 1 ($n \notin O$):** $OPT(n) = OPT(n - 1)$ — ignore item n .
- **Case 2 ($n \in O$):** Item n uses weight w_n . The remaining capacity is $W - w_n$. But $OPT(n - 1)$ doesn't tell us the best solution with capacity $W - w_n$ — it only tells us the best solution with the *original* capacity using $n - 1$ items.

Key Insight: We need to know not just which items, but **how much weight capacity remains**. This requires adding a new variable.

7.3 Adding a Variable: The 2D Subproblem

Definition: $OPT(i, w)$

For each $i = 0, 1, \dots, n$ and each integer $0 \leq w \leq W$:

$$OPT(i, w) = \max_{S \subseteq \{1, \dots, i\}, \sum_{j \in S} w_j \leq w} \sum_{j \in S} w_j \quad (7.1)$$

This is the maximum total weight achievable using a subset of items $\{1, \dots, i\}$ with maximum allowed weight w .

7.4 The Recurrence Relation

Theorem 6.8 (Subset Sum Recurrence)

For all $i = 1, 2, \dots, n$ and all $w = 0, 1, \dots, W$:

If $w < w_i$:

$$OPT(i, w) = OPT(i - 1, w) \quad (6.8a)$$

Otherwise:

$$OPT(i, w) = \max (OPT(i - 1, w), w_i + OPT(i - 1, w - w_i)) \quad (6.8b)$$

Proof (by case analysis):

Consider the optimal solution for the first i items with weight limit w .

Case 1: $w < w_i$

Item i cannot be included (it's too heavy). The optimal solution uses only items 1 through $i - 1$:

$$OPT(i, w) = OPT(i - 1, w)$$

Case 2: $w \geq w_i$

We have two choices for item i :

- **Exclude item i :** Best we can do is $OPT(i - 1, w)$.
- **Include item i :** We gain w_i weight, and have remaining capacity $w - w_i$ for items 1 through $i - 1$. Best value: $w_i + OPT(i - 1, w - w_i)$.

We take the maximum of these two options. ■

7.5 The Algorithm

Algorithm: Subset-Sum

```
Subset-Sum(n, W)
  Array M[0 ... n, 0 ... W]
  Initialize M[0, w] = 0 for each w = 0, 1, ..., W
  For i = 1, 2, ..., n
    For w = 0, 1, ..., W
      If w < wi then
        M[i, w] = M[i - 1, w]
      Else
        M[i, w] = max(M[i - 1, w], wi + M[i - 1, w - wi])
      Endif
    Endfor
  Endfor
  Return M[n, W]
```

7.6 Time Complexity Analysis (Full Derivation)

Theorem 6.9: The Subset-Sum algorithm runs in $O(nW)$ time and uses $O(nW)$ space.

Proof of Time Complexity:

Table size: The table M has $(n + 1) \times (W + 1)$ entries.

Time per entry: Computing each entry $M[i, w]$ requires:

- One comparison $w < w_i$: $O(1)$
- At most one addition $w_i + M[i - 1, w - w_i]$: $O(1)$
- One max operation: $O(1)$

Therefore, each entry takes $O(1)$ time.

Total time:

$$T(n, W) = \sum_{i=1}^n \sum_{w=0}^W O(1) = (n + 1)(W + 1) \cdot O(1) = O(nW)$$

Space Complexity: The table M has $(n + 1)(W + 1) = O(nW)$ entries, each storing an integer. Thus:

$$S(n, W) = O(nW)$$

Note: This is a **pseudo-polynomial** algorithm — polynomial in the numeric value W , but exponential in the number of bits needed to represent W (which is $\log W$). If W is encoded in binary, the input size is $O(n \log W)$, making $O(nW) = O(n \cdot 2^{\log W})$ exponential in the input size.

7.7 Visual Walkthrough

Subset Sum DP Table: Items weights {2, 2, 3}, W=6
Green=Include chosen, Yellow=Can't fit (inherited), White=Exclude chosen

	w=0	w=1	w=2	w=3	w=4	w=5	w=6
i=0	0	0	0	0	0	0	0
i=1	0	0	2	2	2	2	2
i=2	0	0	2	2	4	4	4
i=3	0	0	2	3	4	5	5

Figure 7.1 Subset Sum DP table for weights {2, 2, 3} with $W = 6$. Green cells indicate "include" was chosen; yellow indicates the item couldn't fit (value inherited from above).

Detailed Table Trace:

Table 7.1 Subset Sum Execution Trace ($W = 6$, weights {2, 2, 3})

Phase	Item	Weight	Explanation
Initial	—	—	Row 0: all zeros (no items selected)
$i = 1$	$w_1 = 2$	2	For $w \geq 2$: $\max(0, 2 + 0) = 2$. Row 1: [0, 0, 2, 2, 2, 2, 2]
$i = 2$	$w_2 = 2$	2	At $w = 4$: $\max(2, 2 + M[1,2]) = \max(2, 4) = 4$. Row 2: [0, 0, 2, 2, 4, 4, 4]
$i = 3$	$w_3 = 3$	3	At $w = 5$: $\max(4, 3 + M[2,2]) = \max(4, 5) = 5$. Row 3: [0, 0, 2, 3, 4, 5, 5]

Final Answer: $M[3, 6] = 5$ — achieved by items 2 and 3 (weights $2 + 3 = 5$).

7.8 Finding the Solution Set

Theorem 6.10

Given a table M of optimal values, the optimal set S can be found in $O(n)$ time.

Algorithm and Proof:

Trace back from $M[n, W]$:

- If $M[i, w] = M[i - 1, w]$, item i is **not** in the set. Move to $M[i - 1, w]$.
- If $M[i, w] \neq M[i - 1, w]$, item i is in the set. Move to $M[i - 1, w - w_i]$.

Each step decreases i by 1, so at most n steps. Time: $O(n)$. ■

8. 0-1 Knapsack (Dynamic Programming)

8.1 Problem Definition

Given n items with weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$, and a knapsack capacity W , select a subset of items to **maximize total value** without exceeding weight capacity. Unlike fractional knapsack, items cannot be divided.

8.2 The Recurrence Relation

Theorem 6.11 (0-1 Knapsack Recurrence)

For all $i = 1, 2, \dots, n$ and all $w = 0, 1, \dots, W$:

If $w < w_i$:

$$OPT(i, w) = OPT(i - 1, w) \quad (6.11a)$$

Otherwise:

$$OPT(i, w) = \max(OPT(i - 1, w), v_i + OPT(i - 1, w - w_i)) \quad (6.11b)$$

Proof:

Identical structure to Theorem 6.8 (Subset Sum), but maximizing **value** instead of weight.

Case 1: $w < w_i$

Item i doesn't fit. Must exclude it. $OPT(i, w) = OPT(i - 1, w)$.

Case 2: $w \geq w_i$

- **Exclude:** Best value without item i is $OPT(i - 1, w)$.
- **Include:** Value v_i plus best value from remaining capacity $w - w_i$ using items 1 through $i - 1$: $v_i + OPT(i - 1, w - w_i)$.

Take the maximum. ■

8.3 The Algorithm

Algorithm: 0-1 Knapsack (Dynamic Programming)

```
Initialize array M[0 ... n, 0 ... W]
For w = 0, 1, ..., W
    M[0, w] = 0
Endfor

For i = 1, 2, ..., n
    For w = 0, 1, ..., W
        If w < wi then
            M[i, w] = M[i - 1, w]
        Else
            include_value = vi + M[i - 1, w - wi]
            exclude_value = M[i - 1, w]
            M[i, w] = max(exclude_value, include_value)
        Endif
    Endfor
Endfor

Return M[n, W]
```

8.4 Time Complexity Analysis (Full Derivation)

Theorem 6.12: The 0-1 Knapsack algorithm runs in $O(nW)$ time and uses $O(nW)$ space.

Proof of Time Complexity:

Initialization: Setting $M[0, w] = 0$ for all $w = 0, \dots, W$ takes $O(W)$ time.

Main loops:

- Outer loop: i runs from 1 to $n \rightarrow n$ iterations
- Inner loop: w runs from 0 to $W \rightarrow W + 1$ iterations
- Body operations: one comparison ($w < w_i$), at most one addition, one max $\rightarrow$ all $O(1)$

Therefore:

$$T(n, W) = O(W) + \sum_{i=1}^n \sum_{w=0}^W O(1) = O(W) + n(W + 1) \cdot O(1) = O(nW)$$

Space Complexity: The 2D table has $(n + 1)(W + 1) = O(nW)$ entries.

Space Optimization: Using a **rolling array**, we only need two rows (current and previous) at any time:

$$S_{\text{opt}}(n, W) = O(W)$$

Pseudo-polynomial nature: As with Subset Sum, this is pseudo-polynomial. If W requires $b = \log_2 W$ bits to represent, then the input size is $\Theta(n \cdot b)$, and $O(nW) = O(n \cdot 2^b)$, which is exponential in the input size.

8.5 Visual Walkthrough

0-1 Knapsack DP Table: Items (5,60), (4,50), (6,70), (3,30), W=10
 Green=Include chosen, Yellow=Can't fit (inherited), White=Exclude chosen

	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7	w=8	w=9	w=10
i=0	0	0	0	0	0	0	0	0	0	0	0
i=1	0	0	0	0	0	60	60	60	60	60	60
i=2	0	0	0	0	50	60	60	60	60	110	110
i=3	0	0	0	0	50	60	70	70	70	110	120
i=4	0	0	0	30	50	60	70	80	90	110	120

Figure 8.1 0-1 Knapsack DP table for items {(5,60), (4,50), (6,70), (3,30)} with W=10. Green cells show where "include" gave a better value.

Detailed Trace:

Table 8.1 0-1 Knapsack Execution Trace

Item	Weight	Value	Key Computations
1	5	60	For $w \geq 5$: $\max(0, 60+0) = 60$. Row 1: [0,0,0,0,0,60,60,60,60,60,60]
2	4	50	At $w=4$: $\max(0, 50+0) = 50$. At $w=9$: $\max(60, 50+60) = 110$. Row 2: [0,0,0,0,50,60,60,60,60,110,110]
3	6	70	At $w=6$: $\max(60, 70+0) = 70$. At $w=10$: $\max(110, 70+50) = 120$. Row 3: [0,0,0,0,50,60,70,70,70,110,120]
4	3	30	At $w=3$: $\max(0, 30+0) = 30$. At $w=7$: $\max(70, 30+60) = 90$. Row 4: [0,0,0,30,50,60,70,90,90,110,120]

Final Answer: $M[4, 10] = 120$.

8.6 Tracing the Solution

Algorithm: Trace-Solution

```

Trace-Solution(i, w)
  If i = 0 then
    Output nothing
  Else
    If w < w_i then
      Output the result of Trace-Solution(i - 1, w)
    Else if v_i + M[i - 1, w - w_i] > M[i - 1, w] then
      Output i together with Trace-Solution(i - 1, w - w_i)
    Else
      Output the result of Trace-Solution(i - 1, w)
    Endif
  Endif
End

```

Trace from $M[4, 10] = 120$:

- At (4, 10): $w_4 = 3 \leq 10$. Check: $v_4 + M[3, 7] = 30 + 70 = 100 \not> M[3, 10] = 120$. Skip item 4. $\rightarrow$ Trace-Solution(3, 10)
- At (3, 10): $w_3 = 6 \leq 10$. Check: $v_3 + M[2, 4] = 70 + 50 = 120 > M[2, 10] = 110$. Include item 3. $\rightarrow$ Trace-Solution(2, 4)
- At (2, 4): $w_2 = 4 \leq 4$. Check: $v_2 + M[1, 0] = 50 + 0 = 50 > M[1, 4] = 0$. Include item 2. $\rightarrow$ Trace-Solution(1, 0)
- At (1, 0): $w_1 = 5 > 0$. Skip. $\rightarrow$ Trace-Solution(0, 0)
- Done.

Optimal Set: $\{2, 3\}$ with total weight $4 + 6 = 10$ and total value $50 + 70 = 120$. ✓

9. Principles of Dynamic Programming

9.1 The Three Essential Properties

To develop a dynamic programming algorithm, one needs a collection of subproblems satisfying three properties:

Property (i): Polynomial Number of Subproblems

There are only a polynomial number of distinct subproblems.

- Weighted Interval Scheduling: $n + 1$ subproblems ($OPT(0)$ through $OPT(n)$)
- Subset Sum: $(n + 1)(W + 1) = O(nW)$ subproblems
- 0-1 Knapsack: $(n + 1)(W + 1) = O(nW)$ subproblems

Property (ii): Original Problem Computable from Subproblems

The solution to the original problem can be easily computed from the solutions to the subproblems. Often, the original problem **is** one of the subproblems.

- Weighted Interval Scheduling: $M[n]$ directly gives $OPT(n)$
- Subset Sum: $M[n, W]$ directly gives the optimal value

Property (iii): Natural Ordering + Recurrence

There is a natural ordering on subproblems from "smallest" to "largest," and an easy-to-compute recurrence that determines the solution to a subproblem from solutions to strictly smaller subproblems.

- Weighted Interval Scheduling: Order $0, 1, 2, \dots, n$; recurrence uses $p(j) < j$ and $j - 1 < j$
- Subset Sum: Order by increasing i and w ; each entry depends only on previous row ($i - 1$)

9.2 Memoization vs Iteration

Table 9.1 Memoization vs Iteration over Subproblems

Aspect	Memoization	Iteration
Definition	Saving computed values to avoid recomputation	Systematically computing all subproblems in order
Role in DP	Technique to optimize recursive algorithms	The fundamental DP paradigm
When to use	When recursion structure is natural	When explicit ordering makes reasoning clearer
Overhead	Recursion stack overhead	No recursion overhead
Subproblems solved	Only those needed (on-demand)	All subproblems systematically

9.3 The Design Process: Chicken-and-Egg Problem

Finding the right subproblems and recurrence is often a back-and-forth process. Two strategies exist:

Strategy A: Start with subproblems, find recurrence.

- Formulate a set of subproblems that looks natural.
- Then figure out a recurrence linking them together.

Strategy B: Start with recurrence, determine subproblems.

- Define a recurrence by reasoning about the structure of an optimal solution.
- Then determine which subproblems are necessary to unwind the recurrence.

Weighted Interval Scheduling followed Strategy B

First reasoned about optimal solution structure (include/exclude j), derived recurrence (6.1), then the subproblems ($OPT(0), \dots, OPT(n)$) naturally emerged.

9.4 When to Use DP vs Greedy

Table 9.2 Choosing Between Greedy and Dynamic Programming

Characteristic	Use Greedy	Use Dynamic Programming
Problem structure	Greedy-choice property holds	Optimal substructure + overlapping subproblems
Subproblems after choice	Only one subproblem remains	Multiple overlapping subproblems
Need to compare	No need to compare include vs exclude globally	Must compare solutions to subproblems
Examples	Interval scheduling, fractional knapsack	Weighted interval scheduling, 0-1 knapsack, subset sum
Time complexity	Faster: $O(n \log n)$ or $O(n)$	Slower: $O(nW)$, $O(n^2)$, etc.

Summary: Algorithm Comparison

Table S.1 Complete Algorithm Comparison

Algorithm	Time Complexity	Space Complexity	Type	Key Technique
Interval Scheduling (Greedy)	$O(n \log n)$	$O(n)$	Greedy	Earliest finish time first
Interval Partitioning	$O(n \log n)$	$O(n)$	Greedy	Assign smallest available label
Fractional Knapsack	$O(n \log n)$	$O(n)$	Greedy	Highest value density first
Recursive Weighted Interval	$O(2^n)$	$O(n)$	Recursive	Include/exclude recurrence
Memoized Weighted Interval	$O(n)$	$O(n)$	DP (Memo)	Store OPT(j) in array M
Iterative Weighted Interval	$O(n)$	$O(n)$	DP (Iter)	Bottom-up fill of M
Find-Solution (WIS)	$O(n)$	$O(n)$	Backtrack	Trace through M array
Subset Sum	$O(nW)$	$O(nW)$ or $O(W)$	DP (2D)	Add weight variable
0-1 Knapsack	$O(nW)$	$O(nW)$ or $O(W)$	DP (2D)	Maximize value with weight constraint
Trace-Solution (Knapsack)	$O(n)$	$O(n)$	Backtrack	Trace through 2D table

Key Theorems Reference

Table S.2 Key Theorems and Recurrences

Theorem	Statement
4.1	The greedy set A is compatible
4.2	Stays Ahead: $f(i_r) \leq f(j_r)$ for all $r \leq k$
4.3	Greedy algorithm returns an optimal set
6.1	$OPT(j) = \max(v_j + OPT(p(j)), OPT(j - 1))$
6.2	Item j is in optimal solution iff $v_j + OPT(p(j)) \geq OPT(j - 1)$
6.3	Compute-Opt correctly computes $OPT(j)$
6.4	M-Compute-Opt runs in $O(n)$ time
6.5	Find-Solution returns optimal solution in $O(n)$ time
6.8	Subset Sum recurrence with 2D subproblems
6.9	Subset-Sum algorithm is correct and runs in $O(nW)$
6.10	Optimal set can be found in $O(n)$ time from table
6.11	0-1 Knapsack recurrence (maximize value)
6.12	0-1 Knapsack can be solved in $O(nW)$ time

Final Summary: What Makes DP Work

1. **Optimal Substructure:** The optimal solution to the main problem contains optimal solutions to subproblems.
2. **Overlapping Subproblems:** The same subproblems are solved repeatedly in a naive recursive approach.
3. **Memoization or Iteration:** Store solutions to subproblems to avoid redundant computation.
4. **Polynomial Subproblems:** The number of distinct subproblems must be polynomially bounded.
5. **Natural Ordering:** There must be an ordering that allows building solutions from smaller to larger.